

Q1. Scenario: You are working as a quality analyst comparing distributions across groups and checking for unusual (outlier) values.

Task: Write a program in Python or R to simulate monthly solar-output data for 5 renewable-energy plants and construct boxplots to compare the distributions.

Python code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# -----
# Step 1: Generate simulated dataset
# -----
np.random.seed(42) # For reproducible results

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

plant_a = np.random.normal(420, 20, 12)
plant_b = np.random.normal(450, 25, 12)
plant_c = np.random.normal(400, 30, 12)
plant_d = np.random.normal(470, 18, 12)
plant_e = np.random.normal(430, 22, 12)

# Add a few unusual (outlier) values
plant_b[3] = 540
plant_c[8] = 320
plant_d[10] = 560

# -----
# Step 2: Create DataFrame
# -----
data = pd.DataFrame({
    "Month": months,
    "Plant_A": plant_a.round(2),
    "Plant_B": plant_b.round(2),
    "Plant_C": plant_c.round(2),
    "Plant_D": plant_d.round(2),
    "Plant_E": plant_e.round(2)
})
```

```

# -----
# Step 3: Save dataset
# -----
data.to_csv("solar_output_dataset.csv", index=False)

# -----
# Step 4: Create Boxplot
# -----
plt.figure(figsize=(9, 6))

plt.boxplot(
    [
        data["Plant_A"],
        data["Plant_B"],
        data["Plant_C"],
        data["Plant_D"],
        data["Plant_E"]
    ],
    tick_labels=["Plant A", "Plant B", "Plant C", "Plant D", "Plant E"],
    patch_artist=True
)

# -----
# Step 5: Add title and labels
# -----
plt.title("Monthly Solar Output Distribution Across Renewable-Energy Plants")
plt.xlabel("Renewable-Energy Plants")
plt.ylabel("Monthly Solar Output (kWh)")

plt.grid(axis='y', linestyle='--', alpha=0.6)

plt.tight_layout()

# -----
# Step 6: Save chart image
# -----
plt.savefig("solar_output_boxplot.png", dpi=300)

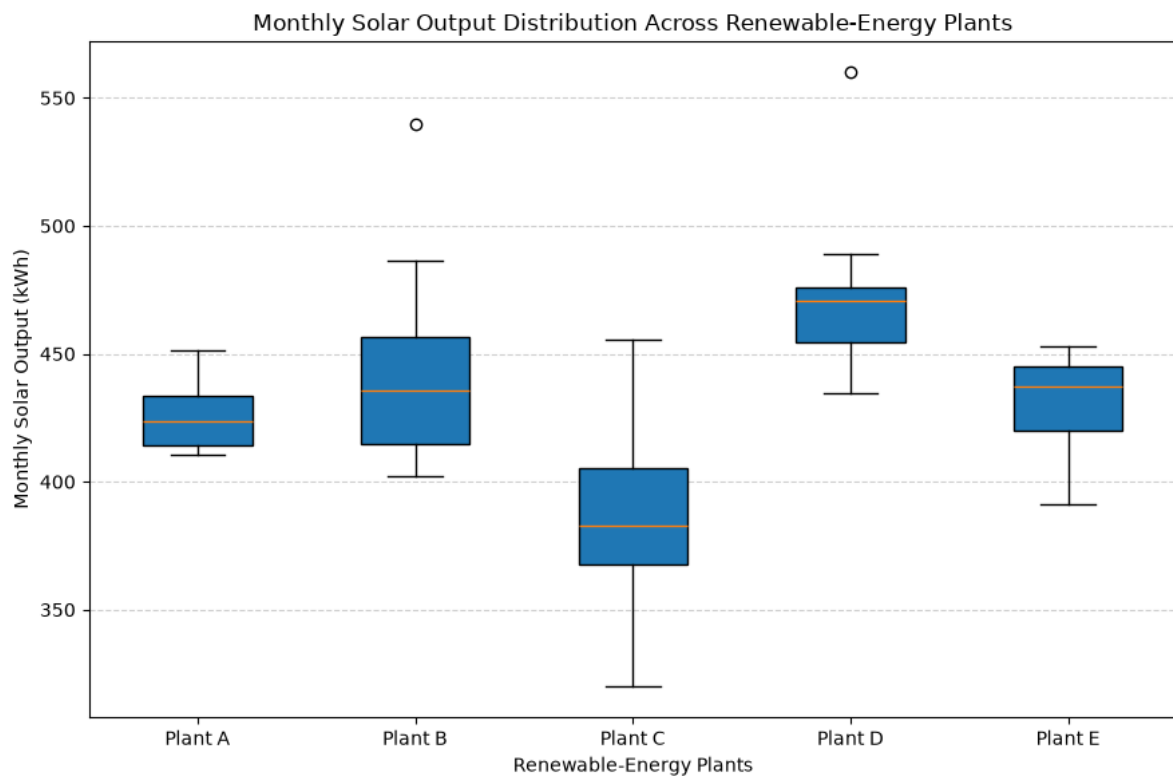
# -----
# Step 7: Display chart

```

```
# -----
plt.show()

print("Dataset saved as: solar_output_dataset.csv")
print("Chart saved as: solar_output_boxplot.png")
```

output:



Dataset:

```
Month,Plant_A,Plant_B,Plant_C,Plant_D,Plant_E
Jan,429.93,456.05,383.67,473.76,437.56
Feb,417.23,402.17,403.33,434.73,391.21
Mar,432.95,406.88,365.47,446.09,437.13
Apr,450.46,540.0,411.27,473.54,421.53
May,415.32,424.68,381.98,483.29,415.11
Jun,415.32,457.86,391.25,473.08,443.46
Jul,451.58,427.3,381.95,467.92,452.68
Aug,435.35,414.69,455.57,464.58,450.49
Sep,410.61,486.64,320.0,443.39,411.54
Oct,430.85,444.36,368.27,457.04,423.2
Nov,410.73,451.69,424.68,560.0,437.29
Dec,410.69,414.38,363.37,489.03,451.46
```

Q2. Scenario: You are working as a business analyst at a retail company that wants to understand and compare key business figures.

Task: Write a program in Python or R to visualize the number of satellites currently operated by 5 different space agencies, using a function that takes the data frame and column names as parameters and returns the plot object.

Python code:

Q2: Number of Satellites Operated by Space Agencies

Aim: Visualize the number of satellites using a reusable plotting function.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# Step 1: Create dataset
```

```
# -----
```

```
data = pd.DataFrame({
```

```
    "Space_Agency": [
```

```
        "NASA (USA)",
```

```
        "CNSA (China)",
```

```
        "Roscosmos (Russia)",
```

```
        "ESA (Europe)",
```

```
        "ISRO (India)"
```

```
    ],
```

```
    "Satellites": [180, 320, 170, 110, 95]
```

```
})
```

```
# -----
```

```
# Step 2: Save dataset
```

```
# -----
```

```

data.to_csv("space_agencies_dataset.csv", index=False)

# -----

# Step 3: Function to create bar chart

# Parameters:

# df    -> DataFrame
# x_col  -> Column for X-axis
# y_col  -> Column for Y-axis

# Returns:

# Matplotlib Figure object

# -----

def plot_satellites(df, x_col, y_col):

    fig, ax = plt.subplots(figsize=(9, 6))

    ax.bar(df[x_col], df[y_col])

    ax.set_title("Number of Satellites Operated by Space Agencies")
    ax.set_xlabel("Space Agencies")
    ax.set_ylabel("Number of Satellites")

    ax.grid(axis='y', linestyle='--', alpha=0.6)

    # Display values above bars
    for i, value in enumerate(df[y_col]):
        ax.text(i, value + 5, str(value), ha='center')

    plt.xticks(rotation=15)

```

```
plt.tight_layout()
```

```
return fig
```

```
# -----
```

```
# Step 4: Generate plot
```

```
# -----
```

```
fig = plot_satellites(data, "Space_Agency", "Satellites")
```

```
# -----
```

```
# Step 5: Save chart
```

```
# -----
```

```
fig.savefig("space_agencies_bar_chart.png", dpi=300)
```

```
# -----
```

```
# Step 6: Display chart
```

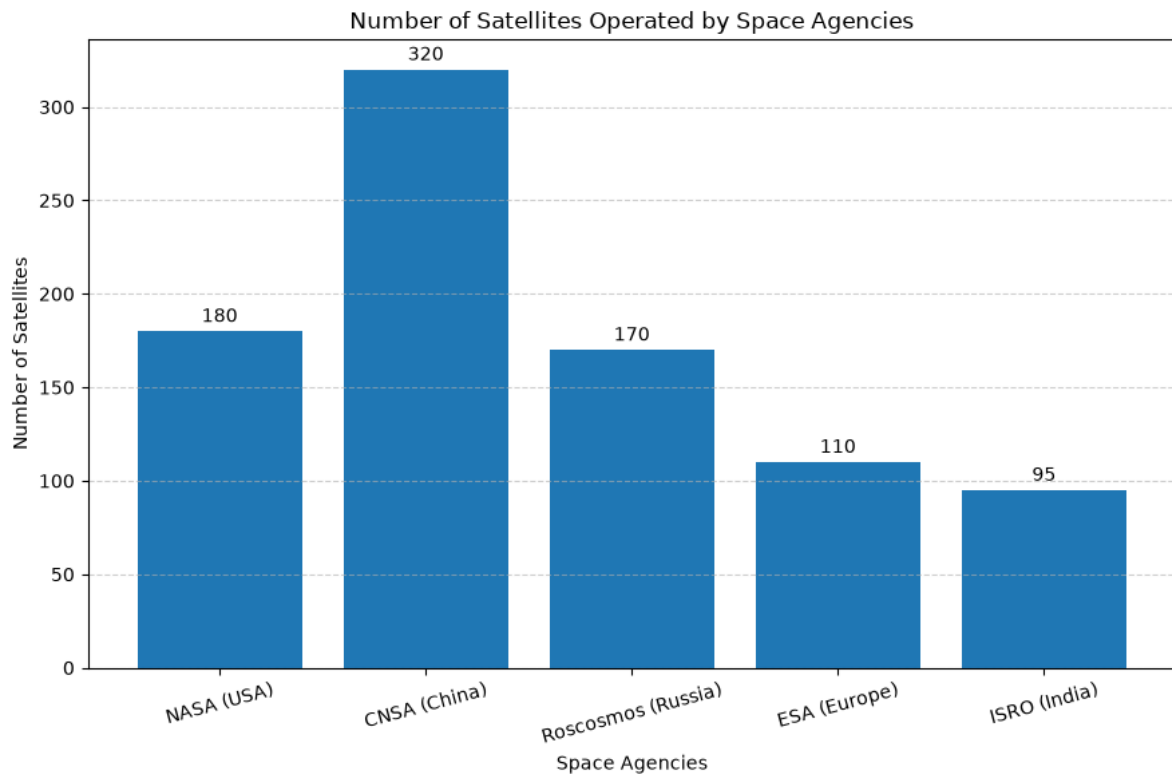
```
# -----
```

```
plt.show()
```

```
print("Dataset saved as: space_agencies_dataset.csv")
```

```
print("Chart saved as: space_agencies_bar_chart.png")
```

```
output:
```



Dataset:

Space_Agency,Satellites

NASA (USA),180

CNSA (China),320

Roscosmos (Russia),170

ESA (Europe),110

ISRO (India),95

Q3. Scenario: You are working as a data analyst who has been asked to study how a particular numeric measurement varies across a group.

Task: Write a Python or R script to simulate the response times (seconds) of a customer-support chatbot for 35 queries using an exponential/gamma distribution and plot the resulting distribution.

Python code:

```
# Q3: Distribution of Customer-Support Chatbot Response Times
```

```
# Aim: Simulate response times using an exponential distribution
```

```
# and visualize the distribution.
```

```
import numpy as np
```

```

import pandas as pd

import matplotlib.pyplot as plt

# -----

# Step 1: Generate simulated response-time data
# -----

np.random.seed(42) # Ensures reproducible results

# Generate 35 response times (in seconds)
response_times = np.random.exponential(scale=4, size=35)

# Round values to 2 decimal places
response_times = np.round(response_times, 2)

# -----

# Step 2: Create DataFrame
# -----

data = pd.DataFrame({
    "Query_No": range(1, 36),
    "Response_Time_Seconds": response_times
})

# -----

# Step 3: Save dataset
# -----

data.to_csv("chatbot_response_times.csv", index=False)

# -----

```



```
# Step 4: Plot Histogram
```

```
# -----
```

```
plt.figure(figsize=(8,6))
```

```
plt.hist(
```

```
    data["Response_Time_Seconds"],
```

```
    bins=8,
```

```
    edgecolor="black"
```

```
)
```

```
# -----
```

```
# Step 5: Add title and labels
```

```
# -----
```

```
plt.title("Distribution of Chatbot Response Times")
```

```
plt.xlabel("Response Time (Seconds)")
```

```
plt.ylabel("Number of Queries")
```

```
plt.grid(axis='y', linestyle='--', alpha=0.6)
```

```
plt.tight_layout()
```

```
# -----
```

```
# Step 6: Save chart image
```

```
# -----
```

```
plt.savefig("chatbot_response_distribution.png", dpi=300)
```

```
# -----
```

```
# Step 7: Display chart
```

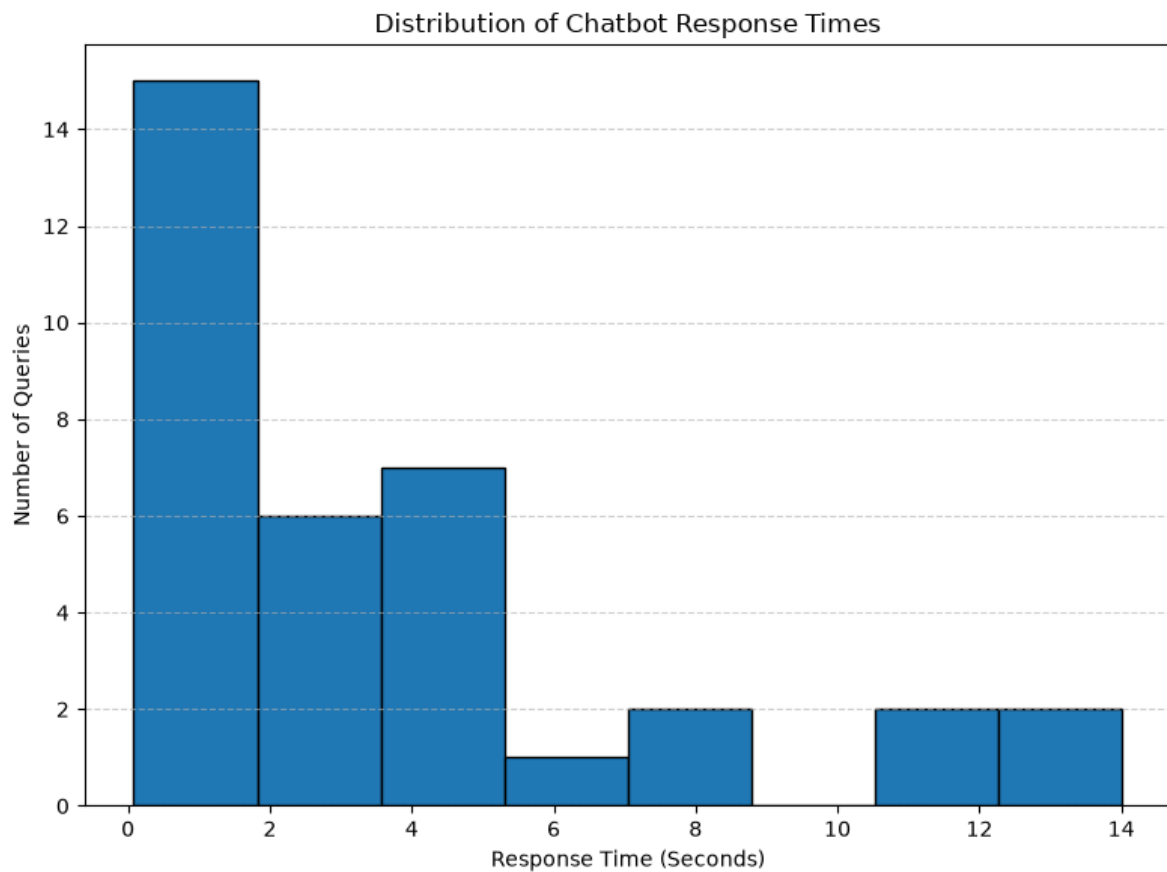
```
# -----
```

```
plt.show()
```

```
print("Dataset saved as: chatbot_response_times.csv")
```

```
print("Chart saved as: chatbot_response_distribution.png")
```

output:



Dataset:

Query_No,Response_Time_Seconds

1,1.88

2,12.04

3,5.27

4,3.65

5,0.68

6,0.68

7,0.24

8,8.04

9,3.68

10,4.93
11,0.08
12,14.01
13,7.15
14,0.95
15,0.8
16,0.81
17,1.45
18,2.98
19,2.26
20,1.38
21,3.79
22,0.6
23,1.38
24,1.83
25,2.44
26,6.15
27,0.89
28,2.89
29,3.59
30,0.19
31,3.74
32,0.75
33,0.27
34,11.89
35,13.48

Q4. Scenario: You are working as a market research analyst who needs to show management how a whole is divided among its parts.

Task: Write a Python or R script to visualize proportions of a household's monthly streaming-subscription spend across 5 platforms, using a function that validates the percentages sum to 100 before plotting.

Python code:

```
# Q4: Household Monthly Streaming Subscription Spend  
  
# Aim: Visualize spending proportions using a pie chart  
  
# after validating that percentages sum to 100.
```

```
import pandas as pd
```

```

import matplotlib.pyplot as plt

# -----
# Step 1: Create dataset
# -----

data = pd.DataFrame({
    "Platform": [
        "Netflix",
        "Amazon Prime",
        "Disney+",
        "Spotify",
        "YouTube Premium"
    ],
    "Percentage": [35, 25, 15, 10, 15]
})

# -----
# Step 2: Save dataset
# -----

data.to_csv("streaming_subscription_dataset.csv", index=False)

# -----
# Step 3: Function to validate and plot
# -----

def plot_streaming_spend(df, category_col, percentage_col):

    # Validate percentages sum to 100
    total = df[percentage_col].sum()

```

```
if total != 100:
    raise ValueError(
        f"Error: Percentages sum to {total}, but they must sum to 100."
    )
```

```
# Create pie chart
```

```
fig, ax = plt.subplots(figsize=(7, 7))
```

```
ax.pie(
    df[percentage_col],
    labels=df[category_col],
    autopct="%1.1f%%",
    startangle=90
)
```

```
ax.set_title("Household Monthly Streaming Subscription Spend")
```

```
ax.axis("equal") # Makes pie chart circular
```

```
return fig
```

```
# -----
```

```
# Step 4: Generate plot
```

```
# -----
```

```
fig = plot_streaming_spend(
```

```
    data,
```

```
    "Platform",
```

```
    "Percentage"
```

```
)
```

```
# -----
```

```
# Step 5: Save chart
```

```
# -----
```

```
fig.savefig("streaming_subscription_pie_chart.png", dpi=300)
```

```
# -----
```

```
# Step 6: Display chart
```

```
# -----
```

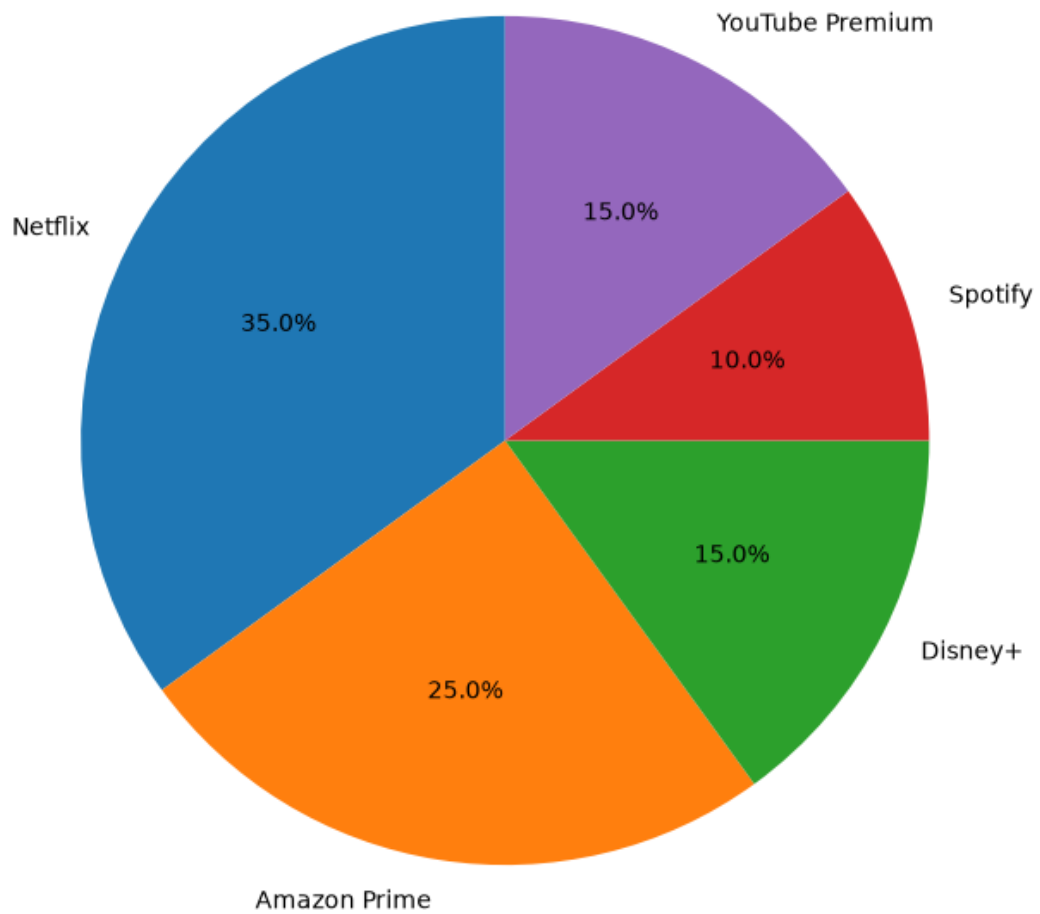
```
plt.show()
```

```
print("Dataset saved as: streaming_subscription_dataset.csv")
```

```
print("Chart saved as: streaming_subscription_pie_chart.png")
```

```
output:
```

Household Monthly Streaming Subscription Spend



Dataset:

Platform,Percentage

Netflix,35

Amazon Prime,25

Disney+,15

Spotify,10

YouTube Premium,15

Q5. Scenario: You are working as a data analyst investigating whether two measured variables are related to one another.

Task: Write a Python or R script to generate podcast episode length (minutes) and average listener retention (%) data for 22 episodes and plot the x-y relationship, adding jitter if points overlap.

Python code:

Q5: Podcast Episode Length vs Listener Retention

Aim: Investigate the relationship between episode length

and average listener retention using a scatter plot.

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# Step 1: Generate simulated dataset
```

```
# -----
```

```
np.random.seed(42) # For reproducible results
```

```
# Episode lengths (minutes)
```

```
episode_length = np.random.randint(15, 61, 22)
```

```
# Listener retention (%) with a slight decreasing trend
```

```
retention = 100 - (episode_length * 0.8) + np.random.normal(0, 5, 22)
```

```
# Keep retention values between 40% and 100%
```

```
retention = np.clip(retention, 40, 100)
```

```
# -----
```

```
# Step 2: Add jitter to avoid overlapping points
```

```
# -----
```

```
length_jitter = episode_length + np.random.uniform(-0.5, 0.5, 22)
```

```
retention_jitter = retention + np.random.uniform(-1, 1, 22)
```



```
# -----  
  
# Step 3: Create DataFrame  
  
# -----  
  
data = pd.DataFrame({  
    "Episode_Length_Minutes": episode_length,  
    "Listener_Retention_Percent": retention.round(2)  
})  
  
  
# -----  
  
# Step 4: Save dataset  
  
# -----  
  
data.to_csv("podcast_episode_data.csv", index=False)  
  
  
# -----  
  
# Step 5: Create Scatter Plot  
  
# -----  
  
plt.figure(figsize=(8, 6))  
  
  
plt.scatter(  
    length_jitter,  
    retention_jitter,  
    s=70  
)  
  
  
# -----  
  
# Step 6: Add title and labels  
  
# -----
```

```
plt.title("Podcast Episode Length vs Average Listener Retention")
```

```
plt.xlabel("Episode Length (Minutes)")
```

```
plt.ylabel("Average Listener Retention (%)")
```

```
plt.grid(True, linestyle='--', alpha=0.6)
```

```
plt.tight_layout()
```

```
# -----
```

```
# Step 7: Save chart image
```

```
# -----
```

```
plt.savefig("podcast_retention_scatter_plot.png", dpi=300)
```

```
# -----
```

```
# Step 8: Display chart
```

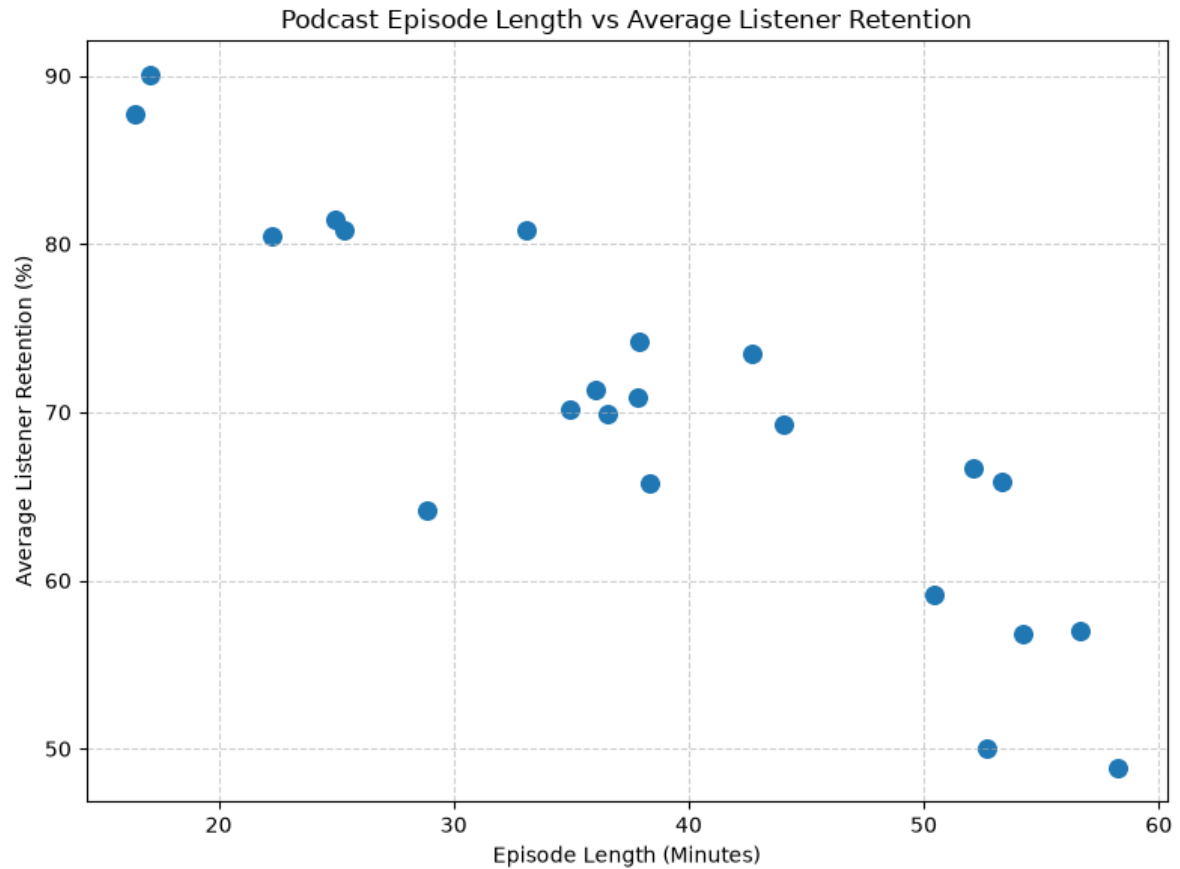
```
# -----
```

```
plt.show()
```

```
print("Dataset saved as: podcast_episode_data.csv")
```

```
print("Chart saved as: podcast_retention_scatter_plot.png")
```

output:



Dataset:

Episode_Length_Minutes,Listener_Retention_Percent

53,64.91

43,73.29

29,64.6

57,57.42

22,81.14

35,71.18

53,50.22

33,81.03

37,70.28

25,81.78

25,82.09

38,73.76

50,58.53

54,56.65

38,70.08

17,89.72

36,70.5

16,87.03

38,65.85

58,49.71

44,69.54

52,66.3